A Unified Framework for Bidirectional English-Hindi Translation and Speech-to-Speech Translation

A B. Tech Project Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Technology

by

Harsh Dahiya
(2201CS30)

under the guidance of

Prof. Rajiv Misra

to the

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY PATNA
PATNA - 800013, BIHAR

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**A Unified Framework for Bidirectional English-Hindi Translation and Speech-to-Speech Translation**" is a bonafide work of **Harsh Dahiya (Roll No. 2201CS30)**, carried out in the Department of Computer Science and Engineering, Indian Institute of Technology Patna under my supervision and that it has not been submitted elsewhere for a degree.*

Harsh Dahiya

May, 2026
Patna.

6/5/2026

Supervisor: **Prof. Rajiv Misra**

Professor & Head of Department,
Department of Computer Science & Engineering,
Indian Institute of Technology Patna, Bihar.

Acknowledgements

Firstly, I would like to thank my project supervisor, Prof. Rajiv Misra, for his guidance and mentorship throughout the duration of this Bachelor of Technology project. His insights and ideas led me to complete this project successfully. I have also submitted a paper in AI-ML systems conference related to the work done in my B.Tech project under his guidance.

I also wish to express my gratitude to the National Supercomputing Mission (NSM) for providing access to the PARAM Rudra Supercomputing infrastructure (NVIDIA A100 80GB).

Finally, I would like to thank the Indian Institute of Technology, Patna, and the Department of Computer Science and Engineering for giving me the academic environment and resources necessary to successfully complete this project and the thesis.

Abstract

As Large Language Models (LLMs) rapidly expand to serve diverse global populations, there is a critical industry shift toward developing sovereign, regional foundation models tailored for morphologically rich and low resource languages. However, the transition from standard generalized architectures to custom localized models often introduces undocumented structural vulnerabilities. Presently, the vast majority of analyses concentrate on zero-shot reasoning from pre-trained models, while the inherent architectural robustness of such models post-training is sorely lacking. As a consequence, when conventional PEFT pipelines and quantization methods are employed on non-optimized regional tensor structures, the resulting models are highly susceptible to catastrophic failures involving gradient explosion, contextual amnesia, and memory capacity violations.

This research aims to determine the efficiency, architectural robustness, and multimodal capability of contemporary decoder-only LLMs specifically trained for Indic languages. A thorough comparative approach will be presented here that assesses the architecture and math robustness of three notable Indic foundation models (Sarvam-1 2B, Pragna-1B, and Param-1 2.9B) on the task of bidirectional machine translation between English and Hindi. The devised strategy is a 1:1 PEFT control experiment using LoRA within BFloat16 precision to isolate architectural weaknesses.

We show through our model evaluations against the uncorrupted IN22-Gen dataset that raw scale of the parameters does not accurately determine the success of the downstream tasks. The analysis performed shows that even though Param-1 has the highest number of parameters, it ends up experiencing a generation failure because of the bugs in the initial configuration for the native Key-Value (KV) cache and the geometry errors of Rotary Positional Embeddings (RoPE). On the other hand, Sarvam-1 performs at close to the top levels of alignment semantics (86.02 COMET). Finally, the optimal model is successfully deployed in a real-time, cascaded Speech-to-Speech Translation (S2ST) system utilizing OpenAI's Whisper Large V3 Turbo and Microsoft Azure Neural TTS. This improved Whisper system guarantees low latency and high precision speech recognition for even the most complicated Indian accent variations, whereas Microsoft Azure neural synthesis generates smooth voice output in Indian accent itself.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

The advent of sovereign, regional Large Language Models has shifted the paradigm of machine translation from dedicated Encoder-Decoder architectures to generalized Causal Language Models. The seminal architecture introduced in *Attention Is All You Need* [1] eliminated recurrence, relying entirely on self-attention mechanisms to map dependencies. However, as these self-attention networks scale and are localized for morphologically rich languages like Hindi, their ability to maintain mathematical integrity during adaptation often degrades.

This paper shifts its attention to **Post-Training Architectural Stability**: assessing the performance of these particular models when subjected to conventional PEFT pipelines and autoregressive decoding. We hypothesize that parameter size is largely irrelevant if a model's foundational tensor geometry, attention mechanisms, and normalization layers are misaligned for standard generation APIs.

1.2 Baseline Paradigms: Encoder-Decoder and Causal

Before working on Indic language models in large scale, preliminary experimentation was performed in order to explore the fundamental properties of traditional architectures in translation tasks:

- **Causal Language Models (GPT-2):** Initial experimentation showed that the conventional Causal LMs without multilingual training face a severe "Embedding Tax." The Byte Pair Encoding (BPE) encoding of GPT-2 fragmented the Devanagari language into pieces. While GPT-2 could generate proper Devanagari characters, the result is semantically nonsensical translation.

- **Multilingual Seq2Seq (mT5-small):** Attempting to train a general 101-language model using a small-scale data set consisting of only 50k sentences turned out to be inefficient. Most of the model's parameters were consumed by an incredibly large vocabulary of size 250k-tokens.

- **Domain-specific Pre-trained Model (Helsinki-NLP / opus-mt-en-hi):** Experimented as a strict zero-shot baseline with a SacreBLEU score of 8.43. Surprisingly enough, enforcing

Beam Search decreased accuracy. It shows that Seq2Seq models have excellent grammatical foundations but a severe domain mismatch compared to modern localized models.

1.3 Contributions

In summary, the key takeaways from our study are as follows:

- **Controlled PEFT Framework:** Our research has developed a rigorous and isolated 1:1 control testing method involving BFloat16 LoRA that allows for a comprehensive analysis of post-training stability of regional LLMs while excluding quantization noise.
- **Architecture Vulnerabilities:** Through our study, we uncover architectural vulnerabilities, including fatal KV cache initialization and RoPE matrix boundary problems in Param-1 (2.9B) in autoregressive inference.
- **Empirical Confirmation of Scale Invariance:** Through IN22-Gen evaluation studies, we have demonstrated evidence for the scale invariant property, which implies that the fine-tuned model is indifferent to the number of parameters, with the smaller Sarvam-1 (2.0B) outperforming the larger Param-1 models on account of superior token fertility and gradient stability.
- **Real-time Deployment of Multimodal S2ST System:** We have successfully built a cascaded speech-to-speech translation pipeline system to showcase the feasibility of fine-tuned models. This is achieved by combining Sarvam-1 with Whisper Large V3 Turbo (for accent-resistant ASR) and Microsoft Azure Neural TTS (for localized prosody).

1.4 Organization of The Report

The subsequent parts of this report are structured as follows: Chapter 2 presents an overview of previous work on Indic machine translation, tokenization, and speech translation pipelines. Chapter 3 covers the dataset used, specifically the corpus and evaluation criteria used to train the model. Chapter 4 explains the methodology adopted, experimental pipelines, and architectural optimizations. Chapter 5 explores the architectural analysis, along with mathematical proof, highlighting weaknesses including the issues of KV Cache Collapse and RoPE Geometry Violation. Chapter 6 describes the quantitative result obtained by evaluating the model based on the criteria for translation baseline metrics. Chapter 7 describes the qualitative findings, examining both context amnesia and semantic competency. Chapter 8 shows multimodal fusion in the end-to-end Speech-to-Speech Translation (S2ST) pipeline.

Chapter 2

Review of Prior Works

2.1 Evolution of Indic Machine Translation

Advancements in the domain of regional NLP systems have led to a paradigm shift away from the use of specialized sequence-to-sequence models towards causal language models. Traditionally, systems like mT5 [12] and IndicTrans2 [2], among others, were considered to provide adequate grammatical coverage. However, these systems have been shown to allocate too many parameters for handling huge multilingual vocabularies and thus suffer from data starvation in terms of regional task performance. The introduction of instruction-tuned models such as Airavata [3] heralded the rise of decoder-only models.

2.2 Tokenization and the Embedding Tax

The core problem that arises from the deployment of generic language models to languages with a morphological structure like Hindi is the ineffectiveness of the traditional BPE. Tokenization research [13] suggests that for languages other than those written in Latin scripts, there is a heavy "embedding tax," where each character is split into several UTF-8 characters. This decrease in semantics and context window size underlines the necessity of having native Indic tokenizers for mathematical stability post-training.

2.3 Parameter-Efficient Fine-Tuning (PEFT)

As the scale of foundational models surpasses memory constraints of conventional hardware devices, the state-of-the-art downstream alignment approach lies in Parameter-Efficient Fine-Tuning (PEFT), more specifically Low-Rank Adaptation (LoRA) [14]. Although the modern trend is predominantly focused on pairing LoRA with aggressive compression (e.g., QLoRA [15]), the latter is extremely efficient in the context of standard transformer geometries. When implemented in custom regional architectures, quantization noise often serves as a catalyst for mathematical collapse; thus, returning to the native precision environment (i.e., BFloat16) becomes necessary to isolate the true limitations of the underlying architecture.

2.4 Attention Mechanisms and Geometric Stability

However, dependence on conventional autoregressive inference pipelines reveal critical weaknesses associated with custom model geometries. Optimizations such as GQA [16] for improved memory bandwidth depend on the initialization state of their KV caches. Moreover, extrapolating sequence length for modern architectures largely depends on RoPE [5]. In situations where custom regional models utilize these approaches with hardcoded assumptions about sequence length or geometry boundaries, tensor misalignment becomes inevitable, an issue that receives limited attention when pre-training is solely considered.

2.5 Speech Translation Pipelines

Traditionally, Speech-to-Speech Translation (S2ST) systems use a cascaded architecture that splits the system into three independent modules: Automatic Speech Recognition (ASR), Machine Translation (MT), and Text-to-Speech (TTS) synthesis. Although direct translational models have been proposed recently, the cascaded architecture is still very efficient due to its strong modularization, which allows integrating state-of-the-art components independently. Recent advances in the area of weakly-supervised, large-scale ASR models [17] have allowed us to significantly reduce the delay in transcription and make the process more excellent for regional accents. Also the move from the conventional concatenative synthesis approach to more advanced neural vocoders [18], results in producing localized prosody that sounds highly natural.

Chapter 3

Dataset Strategy & Benchmarking

The right data pipelines were important to ensure that LLMs did not just remember sentences, but actually learned to generate the language.

3.1 Training Corpus: AI4Bharat Samanantar

Samanantar corpus [22] was used as our fine-tuning data source for the main fine-tuning stage. The reason for using Samanantar is that it is the biggest open-source parallel dataset for Indic languages. It collects vast quantities of data from different domains such as news, official documentation, and web crawling. This data needs to be collected to ensure good instruction fine-tuning of the massive causal language model by giving them a diverse vocabulary.

3.2 Evaluation Benchmark: IN22-Gen

Finally, for evaluation, AI4Bharat IN22-Gen Benchmark [2] was used. IN22-Gen benchmarking is the latest industry benchmarking tool that provides well-verified sentences from humans that rigorously test the model's zero-shot and generalization abilities without contamination. For more details look at the research paper [2] giving proper reasoning for it being the apt benchmark dataset.

3.3 Why IIT Bombay Was Not Used for the Final Comparison

Testing of the initial proofs of concept was conducted using the IIT-Bombay (IIT-B) data. Nonetheless, the IIT-B dataset is outdated and often part of the extensive pre-training datasets used by contemporary LLMs. Conducting evaluations using this dataset exposes high chances of "data leakage," as many LLMs will have learned the test data through previous training. As such, it was not considered in the final comparison framework.

3.4 Evaluation Metrics

The three measures used for assessing the structural and semantic accuracy of the generated translations include:

- **BLEU** [19]: Used in measuring the n-gram precision and lexical accuracy of the generated outputs against the reference text.
- **chrF** [20]: Character level comparison that is especially valuable for morphologically rich scripts such as Devanagari.
- **COMET** [21]: Neural framework that assesses the semantic accuracy and translation quality.

Whereas BLEU and chrF mostly evaluate structural similarity on the surface level, COMET is much more suited for evaluating meaning preservation. This issue was very relevant for this project since there were examples where model generations were very fluent but not accurate from the semantic point of view at all.

Chapter 4

Methodology and Architectural Optimization

All the models were subjected to a strict 1:1 control test to ensure that any possible discrepancy in performance between the architectures was solely due to architectural differences.

4.1 Models Evaluated

The models evaluated are:

- **Sarvam-1 (2B):** regional causal Indic model which has a very efficient tokenization scheme.
- **Pragna-1B (1.25B):** a small model that is decoder-only which serves as the benchmark of efficiency.
- **Param-1 (2.9B):** the largest model among the models tested, but also the least stable model during.

IndicTrans2 was also used as an oracle-style reference point.

4.2 PEFT Configuration

Instead of retraining the complete model, which is, of course, very expensive, we used LoRA (Low-Rank Adaptation) [14] to create lightweight adapters. The configurations used in these include:

- **Rank (r) = 64:** High storage for more complicated syntax recombination.
- **Alpha = 128:** Scaled according to proportionality for gradient stability during backpropagation.
- **Target Modules = all-linear:** Injections of adapters within attention and MLP layers for maximum capacity.
- **Dropout = 0.05:** A 5% dropout was performed to prevent overfitting problems.

- **Precision = Native BFloat16:** Models are trained in native bfloat16 precision from torch. This precision has the same dynamic range as normal 32-bit floats, mathematically ensuring no NaN (under/overflows) crashes in large gradient steps.

4.3 The 4-bit Quantization Breakdown & The Native BF16 Shift

Initially, a 4-bit NF4 quantization technique [15] was employed to reduce the size of the models. While both Pragna-1B and Sarvam-1 benefitted from compression using the above method, Param-1 experienced mathematical instability due to a mathematical collapse. Bitsandbytes depends upon highly optimized binary implementations based on standard geometry of transformers. However, Param-1 makes use of a custom geometry, and thus, using the above mentioned compression algorithm renders the weight matrices unusable, with NaN gradients as a result. Disabling the above and converting to native bfloat16 proved useful.

4.4 Experimental Pipeline Overview

The figure 4.1 provides the full experiment flowchart adopted for the comparative study. It is split into four separate stages. In **Stage 1 (Corpus & Benchmarking)**, the data foundation is set up, employing 100k training pairs from the AI4Bharat Samanantar corpus and the IN22-Gen benchmark for assessment. **Stage 2 (Linguistic Processing)**, the tokenizer mapping is presented, emphasizing the contrasting structure of Sarvam-1's dense native tokenizer against the extremely fragmented BPE/SentencePiece tokenizers of Pragna-1B and Param-1. In **Stage 3 (Foundation Models & Hardware)**, the post-training environment is described, in which all models undergo LoRA fine-tuning natively in torch.bfloat16 precision on NSM PARAM Rudra A100 nodes. Finally, in **Stage 4 (Autoregressive Output & Scoring)**, the standard autoregressive inference flowchart for bidirectional translation is shown, assessed by means of neural semantic alignment (COMET) and strict lexical metrics (SacreBLEU and chrF).

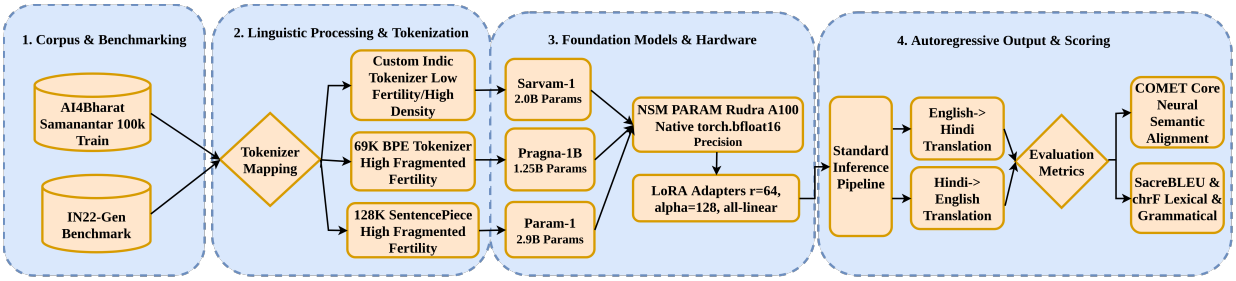


Fig. 4.1 Experimental pipeline to evaluate Indic Foundation Models in an end-to-end manner.

4.5 HPC & MLOps Optimization

Running distributed high-performance computing jobs involved considerable management of the underlying infrastructure:

- **Multi-GPU Spill (CUDA OOM):** Addressed by enforcing hard firewalls for hardware allocation (`export CUDA_VISIBLE_DEVICES=0`) and mapping tensors manually.
- **JIT Compilation Limits:** Absence of the necessary Linux developer headers led to Triton's JIT compiler failing to compile. Isolated compilation chain was added natively to the Conda environment.
- **Exceeded Disk Quotas:** Saving 15GB states in optimizer by the SFTTrainer model exceeded disk capacity.

4.6 Multimodal Integration: S2ST Pipeline

Finally, the optimal LLM (Sarvam-1) is used in a live, real-time Speech-to-Speech Translation pipeline. This gave the true meaning and use to the best fine-tuned LLM.

Speech Input $\rightarrow$ Whisper Large V3 Turbo ASR $\rightarrow$ Sarvam-1 Translation $\rightarrow$ Azure Neural TTS $\rightarrow$ Speech Output

- **ASR (Whisper Large V3 Turbo):** Initially using some simple models, we finally shifted to `openai/whisper-large-v3-turbo` [17]. Due to a high parameter count of 1.5B parameters, it can handle heavy Indian accents. It also gives real-time latency due to its inner mathematics.
- **TTS (Azure Neural):** Using the `edge-tts` library we could integrate Microsoft Azure's Neural TTS API from Text-to-Speech Tasks. It permits us to employ Indian dialects that are locally specific (such as `hi-IN-MadhurNeural`).

Chapter 5

Architectural Analysis and Mathematical Proofs

In order to understand the behavioral anomalies seen in fine-tuning, Table 5.1 provides a summary of the geometric and hyperparameter settings of the base models used in this investigation.

Table 5.1 Architectural Baselines of Evaluated Indic Models

Feature	Param-1 (2.9B)	Sarvam-1 (2.0B)	Pragna-1B (1.25B)
Precision	BFloat16 Mixed	BFloat16 Mixed	BFloat16
Attention	Grouped-Query	Grouped-Query	Grouped-Query
Activation	SiLU	SwiGLU	SiLU
Vocab Size	Custom Indic	Optimized Indic	69,632
Context Window	2,048 Tokens	8,192 Tokens	2,048 Tokens

5.1 Grouped-Query Attention (GQA) and KV Cache Collapse

While official documentation often generalizes foundation model architectures as standard multi-head attention systems, a direct codebase analysis reveals that Param-1 relies on a custom Grouped-Query Attention (GQA) mechanism. Specifically, the architecture divides its processing into 16 primary attention heads and 8 Key-Value (KV) heads to optimize memory bandwidth during generation.

However, this custom implementation introduces a critical initialization flaw when interfacing with standard Hugging Face generative APIs. Autoregressive generation pipelines fundamentally begin at time step $t = 0$ with an empty sequence, meaning the `past_key_values` state is inherently initialized as `NoneType`. However, the Param-1 forward pass ignores the common convention and tries to retrieve the shape of the tensor without any verification by

```python
class ParamBharatGenAttention(nn.Module):
    """Multi-headed attention from 'Attention Is All You Need' paper"""

    def __init__(self, config: ParamBharatGenConfig):
        super().__init__()
        self.config = config
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_attention_heads
        self.head_dim = self.hidden_size // self.num_heads
        self.num_key_value_heads = config.num_key_value_heads
        self.num_key_value_groups = self.num_heads // self.num_key_value_heads
        self.max_position_embeddings = config.max_position_embeddings
        self.rope_theta = config.rope_theta
```

Fig. 5.1 Source code initialization confirming Param-1's native Grouped-Query Attention (GQA) implementation.

```python
    seq_length_with_past = seq_length
    past_key_values_length = 0

    if past_key_values is not None:
        past_key_values_length = past_key_values[0][0].shape[2]
        seq_length_with_past = seq_length_with_past + past_key_values_length

    if position_ids is None:
        device = input_ids.device if input_ids is not None else inputs_embeds.device
```

Fig. 5.2 The forward pass initialization flaw triggering a fatal `NoneType` error during standard autoregressive generation.

`past_key_values[0][0].shape[2]`. This presumption about the existence of the sequence length ($L \geq 1$) leads to an instant mathematical breakdown without generating a single token.

5.2 Cascading RoPE Geometry Violations

After surgical patches for the initial `NoneType` exception have been applied, the erroneous calculations of sequence length will cause a fatal chain reaction in the Rotary Positional Embedding (RoPE).

RoPE uses rotational transformation to transform queries and keys according to their position index P_i within the sequence. As the sequence length variable is compromised due to the problem in the bypassed KV cache, the model produces `position_ids` beyond the allocated range of the cosine and sine matrices in the cache. Consequently, there will be a catastrophic tensor misalignment whenever the `apply_rotary_pos_emb` routine attempts to retrieve non-existent coordinates from the cache matrix. Such a condition will result in a fatal GPU crash, which triggers the CUDA device-side assertion (`Index out of bounds`), making PEFT pipelines mathematically impossible.

```python
def apply_rotary_pos_emb(q, k, cos, sin, position_ids):
    # The first two dimensions of cos and sin are always 1, so we can `squeeze` them.
    cos = cos.squeeze(1).squeeze(0)  # [seq_len, dim]
    sin = sin.squeeze(1).squeeze(0)  # [seq_len, dim]
    cos = cos[position_ids].unsqueeze(1)  # [bs, 1, seq_len, dim]
    sin = sin[position_ids].unsqueeze(1)  # [bs, 1, seq_len, dim]
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed
```

Fig. 5.3 Flaw in tensor allocation due to the overestimation of lengths that force `position_ids` beyond the `cos`/`sin` boundaries.

5.3 Token Fertility and Vocabulary Compression

The efficiency of regional LLMs is heavily dictated by the tokenizer's ability to represent morphologically rich scripts without excessive sub-word fragmentation. Standard multilingual models often suffer from a severe "Embedding Tax," where a single Devanagari character is fragmented into multiple UTF-8 bytes due to an English-centric Byte Pair Encoding (BPE) vocabulary. Sarvam-1 addresses this by utilizing a specialized tokenizer that achieves fertility rates between 1.4 and 2.1 across major Indic languages.

As shown in Fig. 5.4, using Sarvam-1 achieves a 2x to 4x efficiency gain over existing multilingual models. By reducing token-to-word inflation, Sarvam-1 maintains a high density of semantic meaning within its 8,192 token context window. This allows the model to capture deep syntactic dependencies during fine-tuning on the Samanantar corpus, resulting in superior bilingual alignment scores (86.02 COMET) compared to models with higher fragmentation rates.

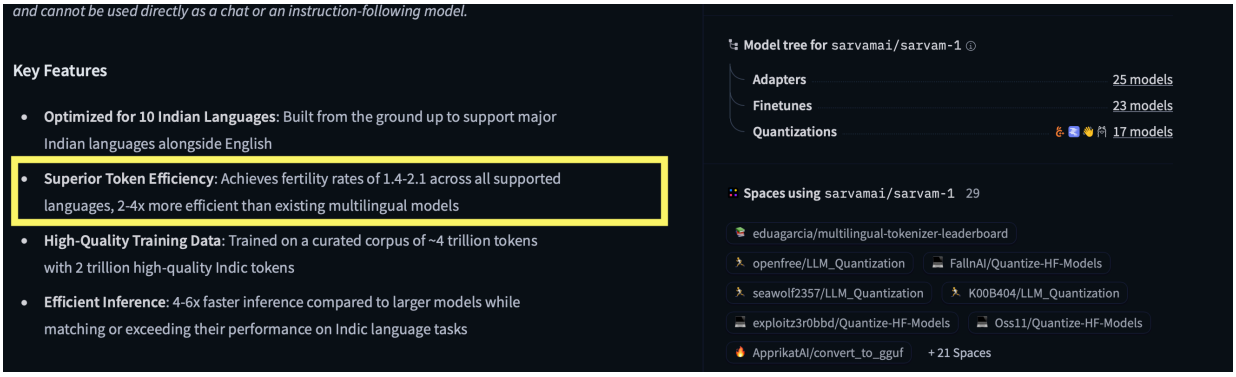


Fig. 5.4 Feature set of Sarvam-1 with 2-4 times faster tokenization than standard multilingual models.

5.4 Normalization and Gradient Stability

In order to solve the issue of gradient instabilities present in custom architectures, Pragna-1B adopts RSNorm (root mean square normalization), as opposed to the traditional LayerNorm. Contrary to conventional approaches, RSNorm pays close attention to rescaling activation vectors alone and ignores the operation of subtracting the mean from each element of the activation vector, hence saving computations while preserving stability. The mathematical formulation of RSNorm on an activation vector x is given by :

$$\text{RSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \gamma \tag{5.1}$$

where an absolute epsilon value of $\epsilon = 1 \times 10^{-5}$ has been used to ensure that there is no mathematical divergence during high-precision training on the NSM PARAM Rudra supercomputer.

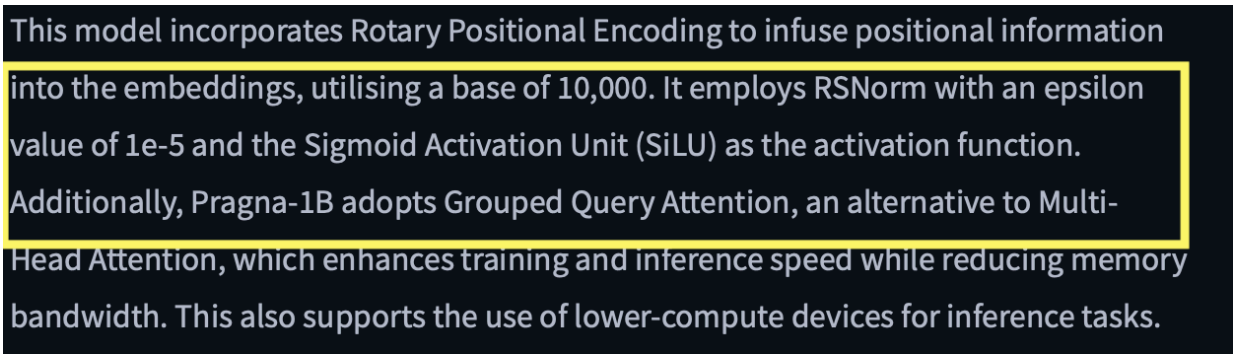


Fig. 5.5 Architecture of Pragna-1B highlighting the implementation of RSNorm and Grouped Query Attention mechanisms.

With this exact stability constant defined, it is ensured that there is no explosion of activations while the gradients are updated massively. The reason for choosing such a strategy is that this algorithm provides stability during BFloat16 fine-tuning without any NaN loss crashes associated with the less-optimal Param-1 tensor geometry.

Chapter 6

Quantitative Results

6.1 Translation Baseline Metrics

Evaluation metrics that capture semantic similarity (COMET) and lexical precision (BLEU, chrF) for translation tasks from English to Hindi and vice versa are presented in Table 6.1 and Table 6.2, along with graphical representations in Fig. 6.1 and Fig. 6.2. IndicTrans2 is provided as an oracle state-of-the-art benchmark.

Table 6.1 English to Hindi Translation (IN22-Gen)

Model Architecture	Params	BLEU	chrF	COMET
IndicTrans2 (Oracle)	∼1.0B	23.40	53.80	82.10
Param-1 (Control)	2.9B	12.19	42.94	59.79
Pragna-1B	1.25B	8.64	32.39	63.62
Sarvam-1	2.0B	19.92	47.67	77.02

6.2 The Pre-Training vs. PEFT Paradox

An important insight generated by the findings of this study is the disconnect between generalized zero-shot benchmark performance and downstream model stability. As noted by Param-1's own model card, the model possesses stable foundational abilities, based on its BLEU performance score of 38.2 on the TruthfulQA-Gen benchmark.

However, as shown in our controlled IN22-Gen experiments, this ability was completely nullified once parametrized via PEFT for autoregressive generation, with BLEU scores of 12.19 (English-to-Hindi) and 7.32 (Hindi-to-English). This clear decline proves beyond a doubt that the

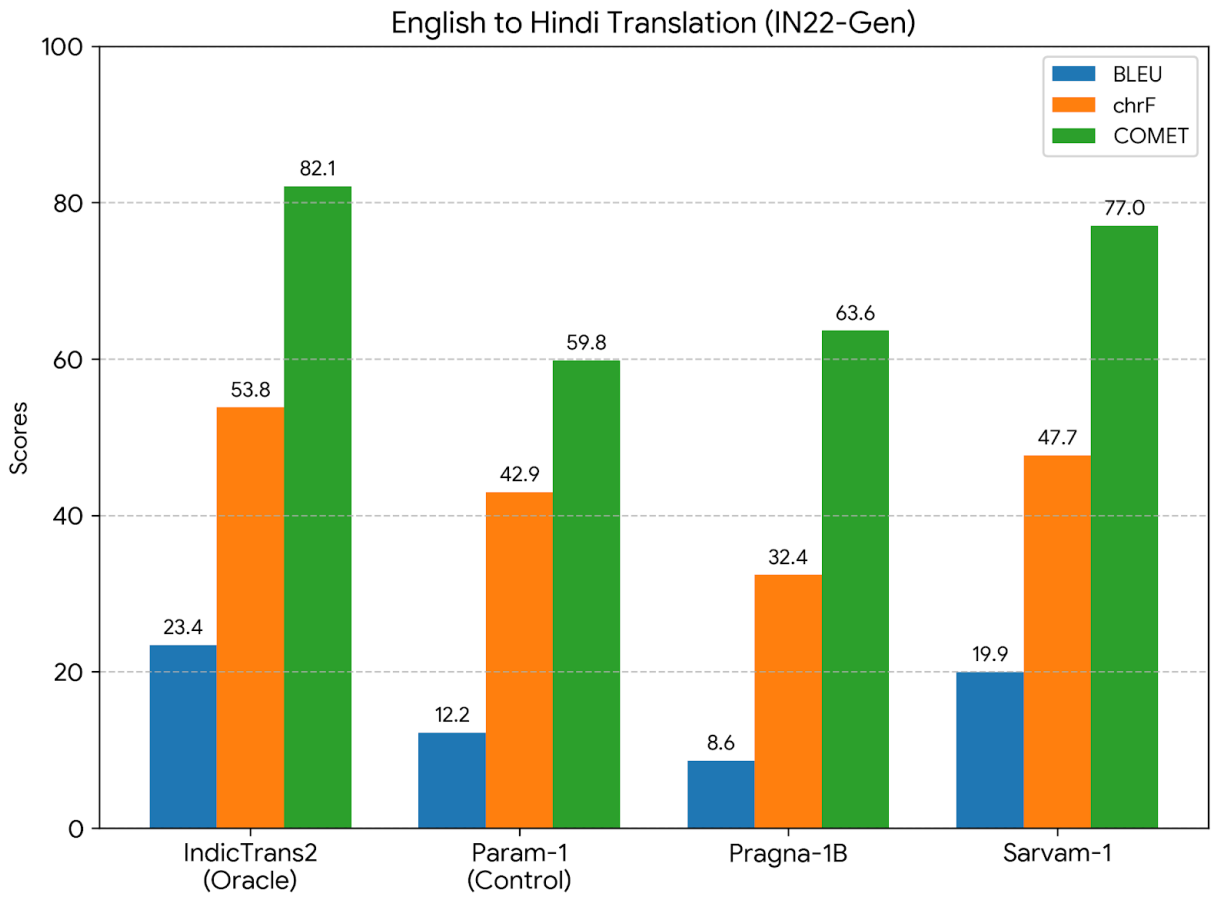


Fig. 6.1 Comparative performance metrics for English to Hindi translation on the IN22-Gen benchmark

Table 6.2 Hindi to English Translation (IN22-Gen)

Model Architecture	Params	BLEU	chrF	COMET
IndicTrans2 (Oracle)	∼1.0B	29.10	57.50	85.40
Param-1 (Control)	2.9B	7.32	19.12	38.19
Pragna-1B	1.25B	18.86	46.10	81.09
Sarvam-1	2.0B	24.37	54.51	86.02

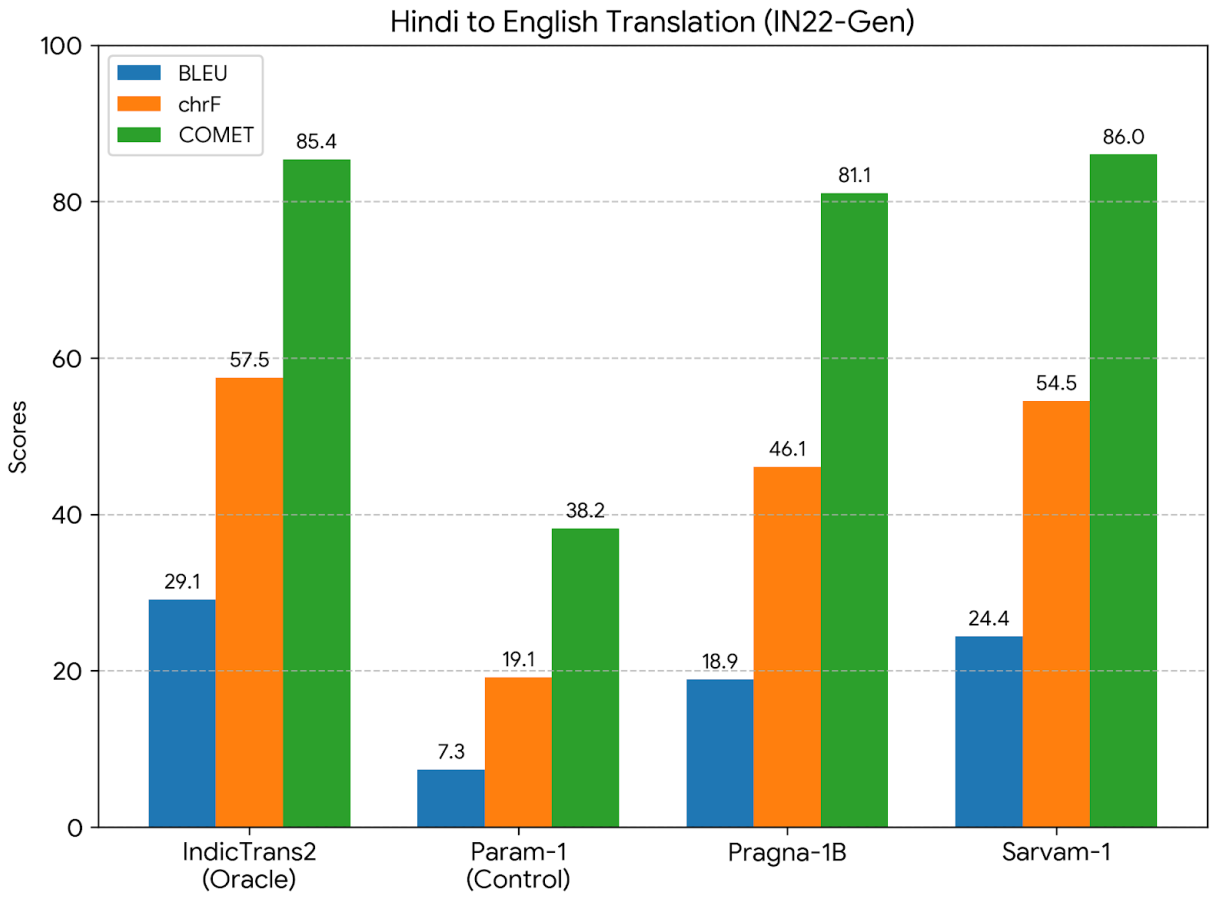
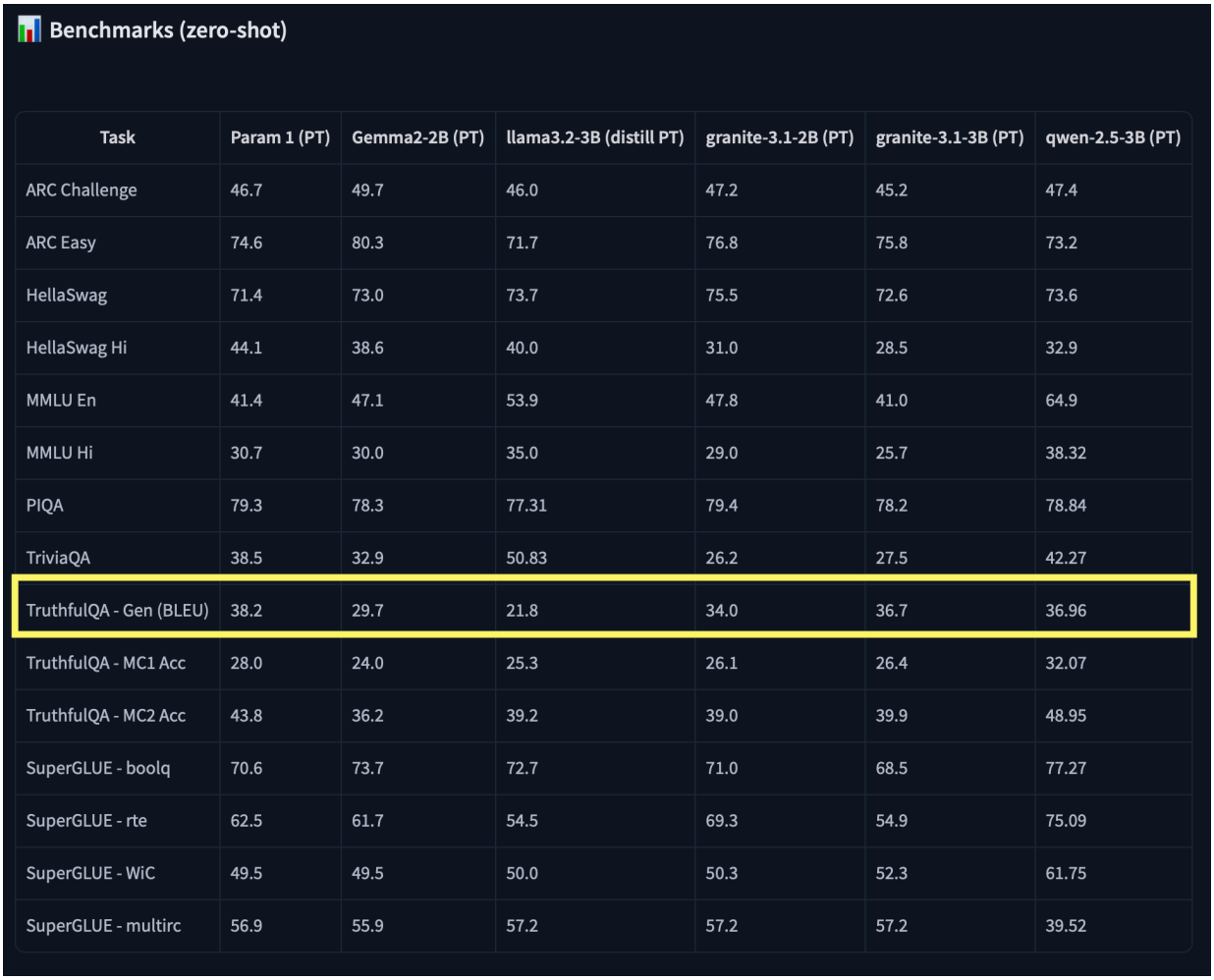


Fig. 6.2 Comparative performance metrics for Hindi to English translation on the IN22-Gen benchmark.



Benchmarks (zero-shot)

Task	Param 1 (PT)	Gemma2-2B (PT)	llama3.2-3B (distill PT)	granite-3.1-2B (PT)	granite-3.1-3B (PT)	qwen-2.5-3B (PT)
ARC Challenge	46.7	49.7	46.0	47.2	45.2	47.4
ARC Easy	74.6	80.3	71.7	76.8	75.8	73.2
HellaSwag	71.4	73.0	73.7	75.5	72.6	73.6
HellaSwag Hi	44.1	38.6	40.0	31.0	28.5	32.9
MMLU En	41.4	47.1	53.9	47.8	41.0	64.9
MMLU Hi	30.7	30.0	35.0	29.0	25.7	38.32
PIQA	79.3	78.3	77.31	79.4	78.2	78.84
TriviaQA	38.5	32.9	50.83	26.2	27.5	42.27
TruthfulQA - Gen (BLEU)	38.2	29.7	21.8	34.0	36.7	36.96
TruthfulQA - MC1 Acc	28.0	24.0	25.3	26.1	26.4	32.07
TruthfulQA - MC2 Acc	43.8	36.2	39.2	39.0	39.9	48.95
SuperGLUE - boolq	70.6	73.7	72.7	71.0	68.5	77.27
SuperGLUE - rte	62.5	61.7	54.5	69.3	54.9	75.09
SuperGLUE - WiC	49.5	49.5	50.0	50.3	52.3	61.75
SuperGLUE - multirc	56.9	55.9	57.2	57.2	57.2	39.52

Fig. 6.3 Official zero-shot results of Param-1 with 38.2 TruthfulQA-Gen BLEU before post-training optimization.

score of a model's zero-shot performance on a pre-training benchmark is not a reliable indicator of performance when the tensor architecture is unstable during PEFT.

Chapter 7

Qualitative Analysis

These quantitative measures are further validated through an intensive qualitative investigation into the generated outputs of each of the models. Through an assessment of some specific translation scenarios-especially those pertaining to specialized terminology and complex sentences-it becomes clear how each model operates.

Task 1: English → Hindi (Medical Advice)
Source: Patients are advised to consult their doctors before starting any new medication or dietary supplements.
Result: इसके अलावा, इस फोन में 12 मेगापिक्सल का कैमरा दिया गया है। इसके अलावा, इस फोन में 12 मेगापिक्सल का कैमरा दिया गया है।
[Error: Severe Hallucination - Smartphone Review Loop]

Task 2: Hindi → English (Medical Advice)
Source: मरीजों को कोई भी नई दवा या आहार पूरक शुरू करने से पहले अपने डॉक्टरों से परामर्श करने की सलाह दी जाती है।
Result: Police reached the spot and brought the situation under control.
[Error: Total Semantic Collapse - News Artifact]

Fig. 7.1 Sample output from Param-1, with loss of context.

7.1 Contextual Amnesia and Hallucination (Param-1)

The Param-1 AI was suffering from severe short-term memory loss during autoregressive generation, falling into disjointed hallucination loops irrespective of translation direction (see Fig. 7.1).

- **Domain Collapse (English to Hindi):** While instructed to translate medical advice on "dietary supplements," the model suddenly lost context mid-way through its task. It appended random smartphone review artifacts from its pre-training data, outputting: "इसके अलावा, इस फोन में 12 मेगापिक्सल का कैमरा दिया गया है।" (Besides this, a 12-megapixel camera is given in this phone).

- **Sensationalist Artifacts (Hindi to English):** In the reverse translation process of the very same medical text, the model underwent semantic collapse, hallucinating news events completely unrelated to the topic: "Police reached the spot and brought the situation under control."

Task 1: English → Hindi (Medical Advice)
Source: Patients are advised to consult their doctors before starting any new medication or dietary supplements.
Result: मरीजों को नए दवाओं या **आहार सब्सिडी** के शुरू करने से पहले अपने डॉक्टर से परामर्श करना चाहिए।
[Error: Vocabulary Miss - 'Dietary Subsidy']

Task 2: Hindi → English (Rain Warning)
Source: भारी बारिश की चेतावनी के बावजूद, वार्षिक उत्सव के लिए मंदिर में **हजारों** भक्त एकत्र हुए।
Result: Despite the heavy downpour, **lakhs of devotees** thronged the temple for the annual festival.
[Error: Magnitude Scaling - Thousands → Lakhs]

Fig. 7.2 Output from Pragna-1B with missed magnitudes and terms.

7.2 Magnitude and Vocabulary Limitations (Pragna-1B)

Producing fluent text, Pragna-1B was able to avoid the problem of extreme hallucinations experienced by Param-1. Nonetheless, the small number of parameters used by the model (1.25B) revealed some weaknesses, especially regarding vocabulary alignment and magnitude alignment issues (see Fig. 7.2).

- **Vocabulary Mismatches:** In translating "dietary supplements" in the English-to-Hindi translation task, Pragna produced the output "आहार सब्सिडी"(dietary subsidy). For the other way round, it produced the output as "food supplements," failing to capture the professional tone demanded by the task.
- **Magnitude Mismatch:** In translating "हजारों" (thousands), Pragna produced the wrong magnitude of translation as "lakhs of devotees" (hundreds of thousands).

7.3 State-of-the-Art Semantic Mastery (Sarvam-1)

Sarvam-1 exhibited remarkable semantic conformance, reproducing the source context with almost complete accuracy in both the target languages (see Fig. 7.3). It showed high levels of bilingual proficiency through the use of highly specialized vocabularies.

- **Vocabulary Precision:** It was able to accurately map the intricate English lexeme "dietary supplements" to the equivalent Hindi medical lexeme "आहार पूरक", and also reverse mapped this in the Hindi-to-English translation scenario.

Task 1: English → Hindi (Medical Advice)
Source: Patients are advised to consult their doctors before starting any new medication or dietary supplements.
Result: मरीजों को सलाह दी जाती है कि वे किसी भी नई दवा या **आहार पूरक** शुरू करने से पहले अपने डॉक्टर से परामर्श करें।
[Success: Precise Medical Vocabulary Alignment]

Task 2: Hindi → English (Medical Advice)
Source: मरीजों को कोई भी नई दवा या आहार पूरक शुरू करने से पहले अपने डॉक्टरों से परामर्श करने की सलाह दी जाती है।
Result: Patients are advised to consult their doctors before starting any new medication or **dietary supplement**.
[Success: Flawless Native-Level Reconstruction]

Fig. 7.3 Outputs from Sarvam-1 with full contextual alignment.

- **Tone:** It accurately preserved numerical magnitudes such as "thousands" and professional tones, including words such as "promising" which it translated to "आशाजनक".

Chapter 8

Multimodal Integration: Speech-to-Speech Translation

8.1 Motivation

Whereas a text-based machine translation system provides essential linguistic competence, a cascade of speech-to-speech translation (S2ST) systems presents a much more directly useful interface that can be easily used by people. The speech-to-speech translation framework enables users to speak in their own voices, to perform interlingual conversation.

8.2 Evolution of Architecture and Logistical Challenges

The initial architectural designs tried to incorporate India-centric APIs to process the audio using endpoints. But this posed some logistic challenges that forced an architectural shift:

- **Vendor Lock-in:** Utilizing proprietary ASR and TTS engines like Gnani.ai models was ruled out since their ecosystem is entirely B2B and commercialized.
- **Administrative Challenges:** The pipeline via Bhashini (Division of Digital India Bhashini), had to be paused indefinitely as the API keys were not issued even after waiting for a long time.

In order to have a working pipeline at no cost, the architecture had to be modified such that there would be no dependency on any third-party APIs. This was done by changing the pipeline to use open-weight models.

8.3 System Architecture

The final pipeline is a three-tier cascaded pipeline using modular design. This is beneficial over end-to-end design, as each component can be evaluated and upgraded independently.

8.3.1 Automatic Speech Recognition (ASR) Layer

Initially, `whisper-small.en` was used for the ASR phase. However, real time testing led to significant accuracy degradation. This was due to the model being sensitive to varying pacing and changing pronunciations based on context in India.

To handle this, the pipeline was shifted to `openai/whisper-large-v3-turbo`. Due to a high parameter count of 1.5B parameters and V3 architecture, it can handle heavy Indian accents. It also gives real-time latency due to its inner mathematics.

8.3.2 Translation Layer

The fundamental semantic routing is performed using the fine-tuned version of **Sarvam-1** that converts the output generated by the ASR module and generates the output in the target language autoregressively. As we have seen in previous chapters, this is because Sarvam-1 has high token fertility and semantic stability properties.

8.3.3 Text-to-Speech (TTS) Layer

For the initial audio output, Google TTS (gTTS) was used, which generated a robotic, emotionless default voice. Due to this, we shifted to Microsoft Azure's Neural TTS API which was integrated through `edge-tts` library.

This allowed us to use localized Indian dialects (e.g., `hi-IN-MadhurNeural` and `en-IN-PrabhatNeural`) which sounded more realistic and familiar. This neural engine also introduces natural breathing pauses and accurate Hindi phonetics.

8.4 Deployment and User Interface

To demonstrate practical applicability, the entire cascaded model was deployed through a Gradio-based web interface using the dynamic allocation of T4 CUDA through Google Colab. This user interface lets users either record their audio or type in the text, and displays the intermediate transcription along with the translated audio output (see Fig. 8.1 and Fig. 8.2).

Live Demonstrations: Video demonstrations showing the working of this S2ST system :

- English to Hindi Translation Pipeline
- Hindi to English Translation Pipeline

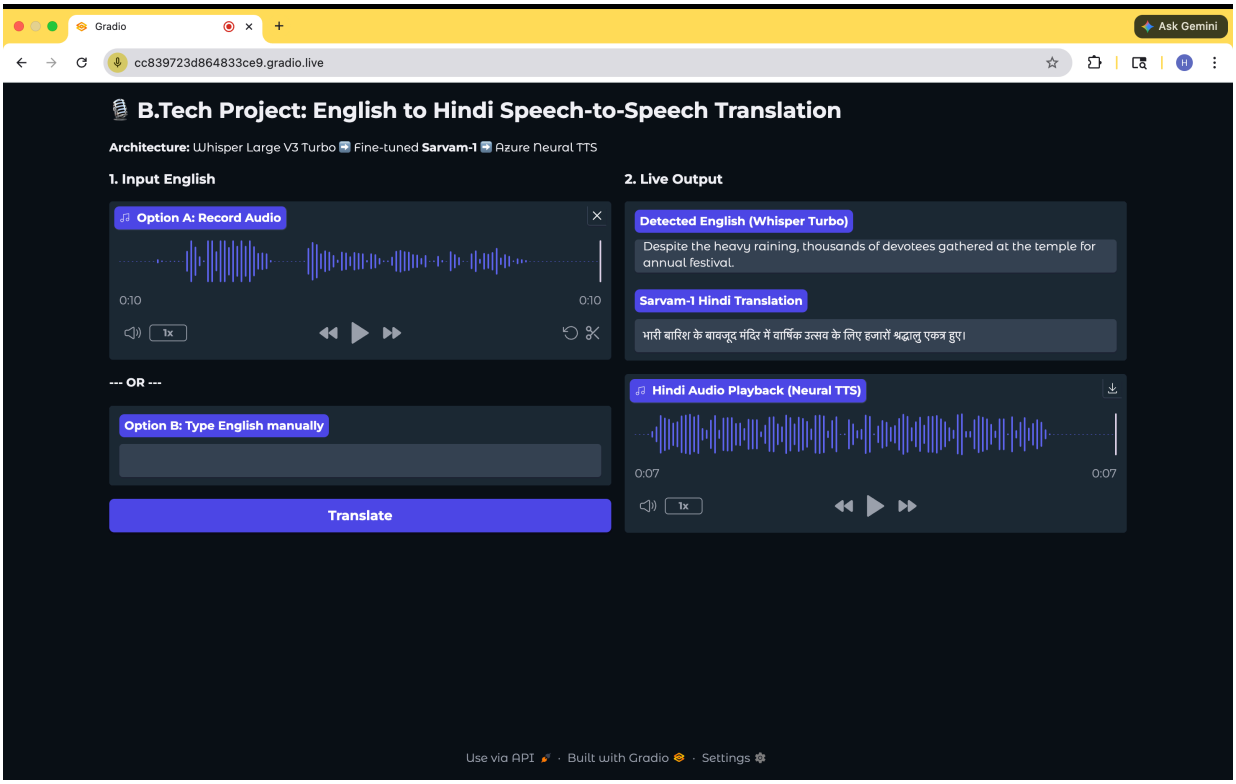


Fig. 8.1 Gradio interface for real-time English to Hindi Speech-to-Speech translation.

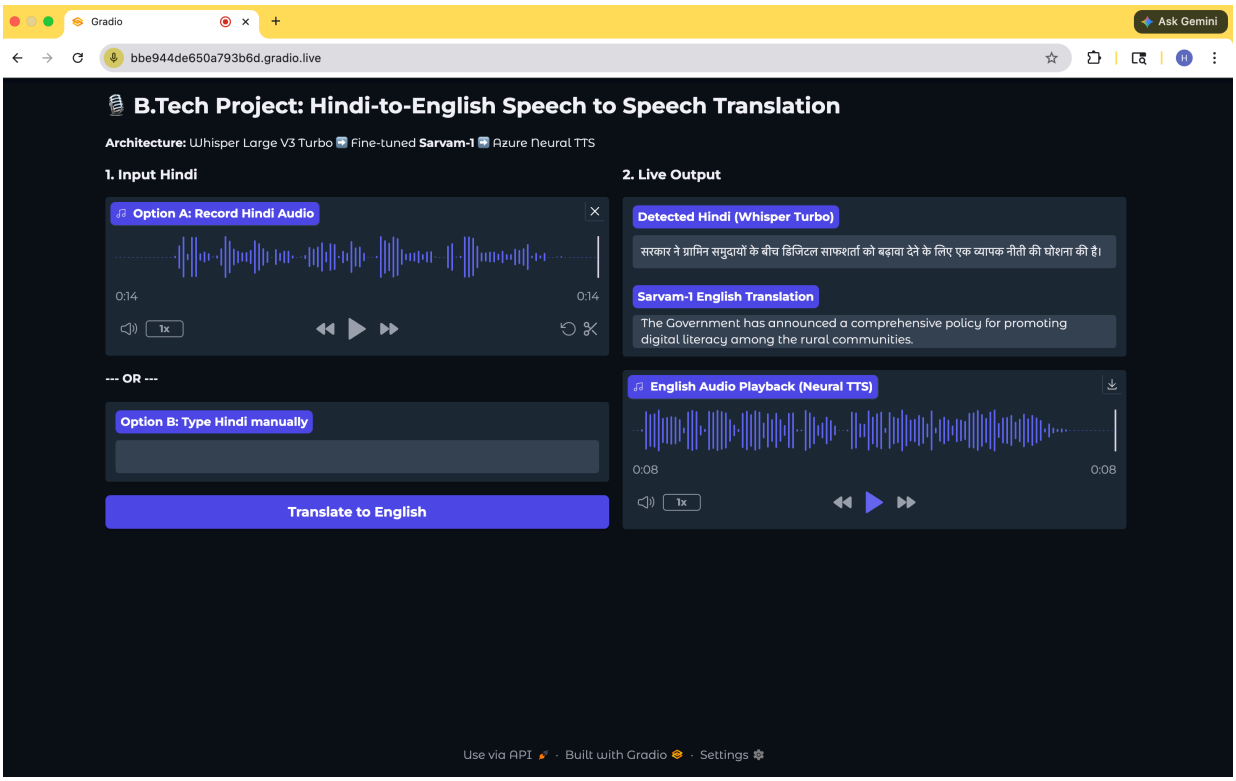


Fig. 8.2 Gradio interface for Hindi to English Speech-to-Speech translation.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This thesis found that English-Hindi bidirectional translation cannot simply be achieved by increasing parameter counts. The findings prove that tokenizer efficiency, cache management, correct positional embedding, and normalization are more important than scaling. Sarvam-1 proved to be the most consistent and practical model for PEFT-based translations in this experiment, whereas Param-1 demonstrated the impact of custom architectural assumptions on common generation workflows.

The extension of the multimodal approach revealed that the translation model could be used in an end-to-end speech-to-speech solution. Using the combination of Whisper Large V3 Turbo, Sarvam-1, and Azure Neural Text to Speech, the entire prototype was successfully created for cross-language voice translation.

9.2 Future Work

A few obvious avenues for future research include:

- extending the experiments to cover other Indian language pairs.
- evaluating the effectiveness of translation prompts fine-tuned on instructions compared to general-purpose translation prompts.
- enhancing resilience to code-mixing and noise.
- switching to a streaming design from a pipelined one.
- developing domain-specific decoding strategies for clinical, legal, and academic texts.

In the long run, one could aim to develop a unified multilingual speech translation system that retains semantics, speaker characteristics, and low latency for multiple Indian languages.

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] J. Gala *et al.*, "IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages," 2023. [Online]. Available: https://arxiv.org/abs/2305.16307

[3] J. Gala *et al.*, "Airavata: Introducing Hindi Instruction-tuned LLM," 2024. [Online]. Available: https://arxiv.org/abs/2401.15006

[4] I. Watts *et al.*, "PARIKSHA: A Large-Scale Investigation of Human-LLM Evaluator Agreement on Multilingual and Multi-Cultural Data," *arXiv preprint arXiv:2406.15053*, 2024. [Online]. Available: https://arxiv.org/abs/2406.15053

[5] J. Su *et al.*, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021. [Online]. Available: https://arxiv.org/abs/2104.09864

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[7] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," 2022. [Online]. Available: https://arxiv.org/abs/2203.15556

[8] Sarvam AI, "Sarvam-1: A 2-Billion Parameter Indic Language Model," 2024. [Online]. Available: https://huggingface.co/sarvamai/sarvam-1

[9] Soket AI Labs, "Pragna-1B: A Multi-lingual Decoder-only Transformer," 2024. [Online]. Available: https://huggingface.co/soketlabs/pragna-1b

[10] K. Pundalik *et al.*, "PARAM-1 BharatGen 2.9B Model," *arXiv preprint arXiv:2507.13390*, 2025. [Online]. Available: https://arxiv.org/abs/2507.13390

[11] BharatGen Team, "Param-1-2.9B-Instruct Model Card and Modeling Implementation," *Hugging Face Repository*, 2025. [Online]. Available: https://huggingface.co/bharatgenai/Param-1-2.9B-Instruct

[12] L. Xue *et al.*, "mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer," *arXiv preprint arXiv:2010.11934*, 2020. [Online]. Available: https://arxiv.org/abs/2010.11934

[13] O. Ahia *et al.*, "Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models," *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13707

[14] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[15] T. Dettmers *et al.*, "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[16] J. Ainslie *et al.*, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," *arXiv preprint arXiv:2305.13245*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13245

[17] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[18] Y. Liu *et al.*, "DelightfulTTS: The Microsoft Speech Synthesis System for Blizzard Challenge 2021," *arXiv preprint arXiv:2110.12612*, 2021. [Online]. Available: https://arxiv.org/abs/2110.12612

[19] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318. [Online]. Available: https://aclanthology.org/P02-1040

[20] M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," in *Proceedings of the Tenth Workshop on Statistical Machine Translation*, 2015, pp. 392–395. [Online]. Available: https://aclanthology.org/W15-3049

[21] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, "COMET: A neural framework for MT evaluation," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 2685–2702. [Online]. Available: https://aclanthology.org/2020.emnlp-main.213

[22] G. Ramesh *et al.*, "Samanantar: The Largest Publicly Available Parallel Corpora Collection for 11 Indic Languages," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 145–162, 2022. [Online]. Available: https://doi.org/10.1162/tacl_a_00452